

THESIS TITLE IN ENGLISH

NAME SURNAME

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
20XX
KMITL-20XX-EN-X-XXX-XXX

THESIS TITLE IN ENGLISH

NAME SURNAME

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
20XX
KMITL-20XX-EN-X-XXX-XXX

Thesis	Thesis Title in English
Student	Mr. Tanakorn Youngmeesuk
Student ID.	65011563
Degree	Bachelor of Engineering
Program	Robotics and AI Engineering
Year	2026
Thesis Advisor	Prof. Dr. Poom Konghuayrob

ABSTRACT IN ENGLISH

A good abstract writing should focus only on the key and interesting aspects of the research. It should emphasize the specific highlights of the research, with the research work being clear, concise, and meeting certain criteria for a good abstract. For example, the word count should be between 200-250 words, or approximately no more than half of an A4 page. Before writing the abstract, one should read and understand their own complete research work to identify various interesting points that can engage general readers, making them want to read the full research. There should be no interpretation or criticism using personal opinions. Complex or aggressive language should be avoided. Local jargon, unnecessary abbreviations, or symbols should not be used as they may lead to misunderstanding. There should be no numerical references, diagrams, tables, various statistical formulas, or equations in the abstract, unless necessary to display analytical results. The abstract may have multiple paragraphs to enhance reader comprehension in each section. Avoid referencing the research work of others in the abstract. The structure of the abstract should also be carefully considered, including font type, font size, page margin, printing standards, and an acceptable format. Proofreading the writing by reading it multiple times will improve the quality of the abstract, making it acceptable for research presentations.

ACKNOWLEDGEMENT

Acknowledgement refers to an announcement of appreciation for virtue. It specifically acknowledges the goodness of individuals who provided assistance leading to the successful completion of research, a thesis, or an academic report. During the process of conducting research, a thesis, or academic work, individuals may receive assistance, guidance, or support that contributes to the successful completion of the task. In recognition of this aid and in gratitude, individuals express their appreciation by writing a Certificate of Appreciation, which is included at the beginning of the research, thesis, or academic work. This serves to demonstrate satisfaction and heartfelt gratitude for the goodness of those who assisted. In essence, a Certificate of Appreciation is a declaration of gratitude.

Name Surname

TABLE OF CONTENTS

	Page
Abstract	ii
ACKNOWLEDGEMENT	iii
TABLE OF CONTENTS	vii
LIST OF TABLES	viii
LIST OF FIGURES	ix
LIST OF ABBREVIATIONS AND SYMBOLS	x
CHAPTER 1 INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Identification	2
1.3 Objectives	3
1.4 Scope and Limitations	3
1.5 Contributions	4
1.6 Thesis Organisation	4
CHAPTER 2 RELATED THEORIES AND LITERATURE REVIEW	5
2.1 ROBOTIC MANIPULATION AND LEARNING-BASED CONTROL	5
2.1.1 Classical Approaches to Robot Control	5
2.1.2 Imitation Learning and Behaviour Cloning	5
2.1.3 End-to-End Visuomotor Policies	5
2.2 VISION-LANGUAGE MODELS	6
2.2.1 Transformer Architecture	6
2.2.2 Vision Encoders: SigLIP and DINOv2	6
2.2.3 Multimodal Language Models and Instruction Following	7
2.2.4 Object Localisation as Spatial Grounding	7
2.3 DIFFUSION MODELS AND FLOW MATCHING	7
2.3.1 Denoising Diffusion Probabilistic Models	7

2.3.2	Flow Matching	8
2.3.3	Diffusion Transformer (DiT)	8
2.4	ACTION TOKENISATION	8
2.4.1	Discrete Action Representations	8
2.4.2	FAST+ Tokeniser	8
2.4.3	Continuous State Representations	9
2.5	RELATED VISION-LANGUAGE-ACTION ARCHITECTURES	9
2.5.1	OpenVLA	9
2.5.2	GR00T N1	9
2.5.3	Diffusion-VLA	10
2.5.4	Knowledge Insulating VLA	10
2.5.5	Summary and Research Gap.....	10
CHAPTER 3	RESEARCH METHODOLOGY AND SYSTEM DESIGN	11
3.1	RESEARCH METHODOLOGY OVERVIEW	11
3.2	SYSTEM ARCHITECTURE	11
3.2.1	Overall Architecture	11
3.2.2	VLM Backbone	12
3.2.3	Vision Expert.....	13
3.2.4	Action Expert	13
3.2.5	State Representation.....	13
3.3	ACTION TOKENISATION	14
3.3.1	FAST+ Discrete Tokenisation	14
3.3.2	Special Action Boundary Tokens.....	14
3.4	PRE-TRAINING CURRICULUM	14
3.4.1	Foundation Model Track: Next-Token Prediction	14
3.4.2	Action Expert Track: Reconstruction and Masked Autoencoding	15
3.5	DATASET COLLECTION PIPELINE.....	16
3.5.1	Hardware Setup	16
3.5.2	Stage 1: Trajectory Recording	16
3.5.3	Stage 2: Trajectory Post-Processing.....	17
3.5.4	Stage 3: Multimodal Dataset Creation.....	17
3.5.5	Stage 4: Smooth Trajectory Execution	17
3.5.6	Dataset Summary and Formatting.....	18
3.6	IMPLEMENTATION DETAILS	18
3.6.1	Hardware and Software.....	18
3.6.2	Optimiser and Learning Rate Schedule	19
3.6.3	Hyperparameter Summary	19
3.7	INFERENCE PROCEDURE	19

5.3.8.5	Visual Materials and Other Media	31
5.3.8.6	Electronic Media.....	31
BIBLIOGRAPHY		32
Appendix A Guidelines and Procedures For Thesis Writing		34
AUTHOR BIOGRAPHY		37

LIST OF TABLES

Table	Page
2.1 Comparison of related VLA architectures. FM = Flow Matching, FS = Frozen Backbone, CT = Continuous Tokenisation, DT = Discrete Tokenisation.....	10
3.1 Data format of a single episode in the collected dataset. $N_{\text{seg}} = 40$, $N_{\text{buf}} = 20$	18
3.2 Key hyperparameters of the proposed VLA architecture.	19

LIST OF FIGURES

Figure		Page
3.1	Overall architecture of the proposed VLA system at a single inference timestep t . The frozen VLM backbone (cyan) processes the instruction and overview image; the Vision Expert (blue) processes the end-effector image; the Action Expert (purple) generates the continuous trajectory conditioned on both upstream modules.....	12

LIST OF ABBREVIATIONS AND SYMBOLS

C	Specific heat
P	Pressure
V	Volume

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The aspiration to endow robotic systems with the capacity for autonomous, general-purpose manipulation has been a defining pursuit of artificial intelligence research for several decades. Classical approaches to robot control, grounded in handcrafted heuristics, explicit geometric planning, and task-specific programming pipelines, have achieved considerable success within narrowly constrained environments. However, these paradigms exhibit a fundamental brittleness: they demand exhaustive engineering effort for each new task, fail to gracefully accommodate distributional shifts in the visual environment, and cannot leverage the rich semantic knowledge embedded in natural language instructions. Consequently, deploying robotic agents in unstructured, real-world settings—where objects, lighting conditions, and task goals vary continuously—remains an open and formidable challenge.

The transformative advances in large-scale deep learning over the past decade have opened a compelling alternative trajectory. The emergence of Vision-Language Models (VLMs) [12, 9], trained on internet-scale multimodal corpora, has demonstrated that neural networks can acquire remarkably robust visual understanding, spatial reasoning, and natural language grounding without task-specific supervision. These models encode not merely perceptual features, but higher-order semantic relationships—an ability that maps intuitively onto the requirements of instruction-following robotic manipulation, where an agent must interpret a command such as *“put the banana in the green box”*, locate the relevant objects in the scene, and execute a coordinated sequence of physical motions to accomplish the goal.

This convergence of insights from natural language processing and robotic control has catalysed the emergence of Vision-Language-Action (VLA) models—a class of architectures that unify visual perception, language comprehension, and continuous motor action generation within a single, end-to-end trainable framework. VLA models have shown early but encouraging results in generalising manipulation policies to novel objects, instructions, and scene configurations, suggesting that scaling the paradigms of foundation model research to the action domain may represent a productive path toward general-purpose robotic intelligence [11, 13].

Nevertheless, the design of effective VLA architectures presents a set of non-trivial technical challenges that the existing literature has only partially resolved. Three interrelated problems are of particular relevance to this thesis. *First*, the choice of action representation—specifically, how continuous robot trajectories are encoded into a form that a language backbone can process—critically influences both the fidelity and the computational tractability of the resulting policy. Discrete tokenisation strategies, such as the FAST+ tokeniser [17], have emerged as a strong baseline, yet the design space of customised tokenisation remains incompletely explored. *Second*, the architectural coupling between the language backbone and the action generation module—whether through discrete token prediction, hidden state conditioning, or feature modulation mechanisms such as Feature-wise Linear Modulation (FiLM) [15]—introduces significant tradeoffs between interpretability, latency, and generalisation capacity. *Third*, the training methodology, including the formulation of pre-training objectives, the choice of auxiliary tasks

such as object localisation, and the role of masked autoencoding for action token reconstruction, remains an active area of investigation with no established consensus on best practice.

These challenges motivate the present work. This thesis proposes and evaluates a novel VLA architecture designed to address the aforementioned limitations through a principled synthesis of current best practices and targeted architectural innovations.

1.2 Problem Identification

The central problem addressed in this thesis concerns the effective integration of a frozen Vision-Language Model backbone with a learned action generation module for instruction-following robotic manipulation. More specifically, the following research gaps have been identified through a systematic review of the relevant literature.

Gap 1: Suboptimal Action Representation in Hybrid Architectures. Existing VLA frameworks predominantly adopt one of two representational strategies: purely discrete action tokenisation, wherein trajectory chunks are mapped to a finite vocabulary and predicted autoregressively, or purely continuous conditioning, wherein the action model is conditioned on dense hidden states extracted from the language backbone. Each strategy entails distinct limitations. Discrete tokenisation introduces quantisation error and constrains temporal resolution, while continuous conditioning requires maintaining full differentiability through the backbone, imposing substantial memory overhead and complicating frozen-backbone deployment. A hybrid approach that selectively combines discrete token prediction with hidden state conditioning based on the action phase has not been rigorously studied.

Gap 2: Inadequate Exploitation of End-Effector Visual Context. Most existing VLA models process visual context through a single shared encoder that receives a global overview image of the workspace. This design conflates information at markedly different spatial scales: the coarse, scene-level context necessary for goal identification and obstacle avoidance, and the fine-grained, proximity-level context necessary for precise grasping and manipulation. The role of a dedicated end-effector camera stream, processed through a specialised vision encoder such as DINOv3 [13], as a complementary input to the action generation module has received insufficient attention in prior architectural studies.

Gap 3: Unclear Pre-Training Curriculum for Unified VLA Systems. The pre-training stage of VLA models typically optimises a next-token prediction objective over instruction-state-action sequences. However, the optimal composition of auxiliary objectives—including object localisation prediction, state description generation, and action token reconstruction via masked autoencoding—and the scheduling of these objectives across training stages remain poorly understood. In particular, the extent to which spatial grounding tasks, such as predicting bounding box coordinates for task-relevant objects, improve downstream manipulation performance through improved spatial reasoning in the language backbone has not been systematically ablated.

Building upon these identified gaps, this thesis poses the following primary research question:

How can a frozen Vision-Language Model backbone be effectively coupled with a diffusion-based action expert and a dedicated visual feature extractor to produce a VLA system that is compu-

tationally efficient, data-effective, and capable of generalising instruction-following manipulation policies to novel objects and scene configurations?

The following secondary questions are also addressed:

- 1) What is the relative contribution of discrete action token conditioning versus hidden state conditioning on the quality and consistency of generated trajectories?
- 2) Does incorporating end-effector image features through a specialised encoder yield measurable improvements in manipulation precision compared to overview-only visual input?
- 3) To what extent does including object localisation as an auxiliary pre-training task improve the spatial reasoning capacity and downstream task performance of the language backbone?

1.3 Objectives

The overarching objective of this research is to design, implement, and empirically evaluate a novel VLA architecture that effectively addresses the research gaps identified in the preceding section. To this end, the following specific objectives are pursued:

- 1) To propose a dual-encoder VLA architecture comprising a frozen VLM backbone (Qwen3 with SigLIP vision encoder, approximately 2 billion parameters), a dedicated end-effector vision expert (DINOv3, approximately 86 million parameters), and a Diffusion Transformer (DiT) action expert (approximately 300 million parameters), conditioned jointly on hidden states from the VLM and visual features from the vision expert.
- 2) To design and implement a two-stage pre-training curriculum in which the foundation model is trained via next-token prediction over a multi-task sequence comprising object localisation coordinates, natural language state descriptions, and FAST+ discrete action tokens, while the action expert is trained via trajectory reconstruction and masked autoencoding loss.
- 3) To conduct ablation studies systematically evaluating the impact of (a) discrete versus hidden state action conditioning, (b) end-effector visual feature inclusion, and (c) object localisation as an auxiliary pre-training task on downstream manipulation performance.
- 4) To benchmark the proposed architecture against established VLA baselines, including Open-VLA [5] and GR00T N1 [10], across a suite of simulated and physical manipulation tasks.

1.4 Scope and Limitations

To maintain clarity of focus, the scope of this research is delimited as follows. The proposed architecture targets table-top robotic manipulation tasks involving a single robotic arm with a parallel-jaw or compliant gripper. The study does not address bi-manual manipulation, mobile locomotion, or multi-agent coordination. Visual observations are assumed to be provided by two calibrated RGB cameras: one providing an overview of the workspace and one mounted at or near the end-effector. Depth sensing and proprioceptive state estimation are considered supplementary inputs and are not the primary focus of the architectural investigation. Pre-training data are drawn from publicly available robot demonstration datasets; data collection procedures and the construction of novel datasets fall outside the scope of this work. Furthermore, while inference latency and model compression are acknowledged as important practical considerations, they are treated as secondary metrics relative to task success.

rate and generalisation performance in the evaluation framework.

1.5 Contributions

The principal contributions of this thesis are as follows:

- 1) A **novel dual-encoder VLA architecture** that decouples the processing of overview and end-effector visual streams through a frozen VLM backbone and a trainable DINOv3-based vision expert respectively, and conditions a DiT-based action expert jointly on VLM hidden states and vision expert features. This architectural decomposition is demonstrated to improve manipulation precision without incurring proportional increases in computational cost.
- 2) A **structured two-stage pre-training curriculum** incorporating object localisation, state description generation, and FAST+ action token prediction as complementary next-token prediction tasks for the foundation model, together with a reconstruction and masked autoencoding objective for the action expert. The curriculum is shown empirically to accelerate convergence and improve generalisation relative to single-objective training baselines.
- 3) A **systematic ablation study** characterising the relative contributions of the discrete action token pathway and the hidden state conditioning pathway to trajectory quality, as well as the incremental benefit of end-effector visual features and spatial grounding auxiliary tasks.
- 4) **Open-source release** of the proposed architecture implementation, pre-training pipeline, and evaluation benchmarks to facilitate reproducibility and further community research.

1.6 Thesis Organisation

The remainder of this thesis is structured as follows. Chapter 2 provides a comprehensive review of the relevant literature, tracing the development of VLA models from their foundational precursors in imitation learning and multimodal language modelling, and critically examining the architectural and training choices made in seminal works including OpenVLA, GR00T N1, and Diffusion-VLA. Chapter 3 presents the proposed architecture in detail, covering the design rationale for each component, the mathematical formulation of the training objectives, and the implementation specifics of the pre-training and fine-tuning pipelines. Chapter 4 describes the experimental setup, datasets, evaluation metrics, and baseline comparisons employed in this study, followed by a thorough presentation and discussion of the empirical results and ablation findings. Chapter 5 synthesises the conclusions drawn from the experimental evidence, revisits the research questions posed in this chapter, acknowledges the limitations of the current work, and identifies promising directions for future investigation.

CHAPTER 2

RELATED THEORIES AND LITERATURE REVIEW

This chapter surveys the theoretical foundations and prior research directly relevant to the design of the Vision-Language-Action (VLA) architecture proposed in this thesis. The review is organised into five principal areas: the evolution of robotic manipulation control paradigms, the architecture and capabilities of Vision-Language Models, the theoretical basis of diffusion and flow-matching action generation, approaches to action tokenisation, and a critical comparative analysis of seminal VLA architectures that directly inform the design decisions presented in Chapter 3.

2.1 ROBOTIC MANIPULATION AND LEARNING-BASED CONTROL

2.1.1 Classical Approaches to Robot Control

Traditional robot manipulation systems are constructed around a modular pipeline comprising perception, state estimation, motion planning, and low-level control subsystems, each of which is engineered independently and integrated through well-defined interfaces. Perception modules employ geometric models and hand-designed feature detectors to identify object poses from depth or RGB sensor data; planning modules invoke algorithms such as Rapidly-exploring Random Trees (RRT) or Probabilistic Roadmaps (PRM) to compute collision-free trajectories in configuration space; and control modules execute these trajectories via Proportional-Integral-Derivative (PID) regulators or model-predictive controllers [6].

These pipelines achieve high reliability within the specific operational envelope for which they are engineered. However, their dependence on explicit world models, manually specified object representations, and task-specific heuristics renders them brittle in the face of distributional change: the introduction of a previously unseen object geometry, an alteration in lighting conditions, or a novel task goal typically necessitates substantial re-engineering. This fundamental limitation motivates the shift toward data-driven, learning-based approaches.

2.1.2 Imitation Learning and Behaviour Cloning

Imitation learning addresses the robot control problem by learning a policy $\pi_{\theta}(\mathbf{a} \mid \mathbf{o})$ that maps observations $\mathbf{o}$ to actions $\mathbf{a}$ by supervised regression against a dataset of expert demonstrations $\mathcal{D} = \{(\mathbf{o}_i, \mathbf{a}_i)\}$. In its simplest instantiation, known as Behaviour Cloning (BC), the policy is trained by minimising the mean squared error between predicted and demonstrated actions [18]. Despite its conceptual simplicity, BC is susceptible to compounding errors arising from covariate shift: small deviations from the demonstration distribution at inference time produce states that are increasingly out-of-distribution, leading to cascading failures. Methods such as DAgger [20] address this by iteratively augmenting the training distribution with on-policy observations relabelled by the expert, though they impose a requirement for interactive expert access that is often prohibitive in physical robot settings.

2.1.3 End-to-End Visuomotor Policies

The introduction of deep convolutional neural networks as visual encoders enabled the development of end-to-end visuomotor policies that directly map raw pixel observations to motor commands without

explicit intermediate representations. Seminal work by Levine et al. [7] demonstrated that such policies could be trained from large collections of robot demonstrations to perform a variety of manipulation tasks, albeit within a restricted range of objects and scene configurations. Subsequent work established that architectural choices, particularly the design of the visual encoder, the action output head, and the temporal context window, significantly influence the generalisation capacity of the resulting policy. This body of work provides the direct precursor to the VLA paradigm, in which the visual encoder and policy backbone are replaced by a pre-trained foundation model of substantially greater representational capacity.

2.2 VISION-LANGUAGE MODELS

2.2.1 Transformer Architecture

The Transformer [21] is the foundational architectural component underlying all modern VLMs. It processes input sequences through a series of multi-head self-attention layers, each of which computes a weighted aggregation of value vectors for every position in the sequence, with attention weights determined by the scaled dot-product between query and key vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (2.1)$$

where Q , K , and V are linear projections of the input, and d_k is the dimension of the key vectors. The self-attention mechanism enables each position to attend to all other positions in the sequence, endowing the model with the capacity to capture long-range dependencies that recurrent architectures struggle to represent efficiently. Position-wise feed-forward sublayers, residual connections, and layer normalisation are interleaved with the attention sublayers to form a complete Transformer block. Large language models are constructed by stacking many such blocks and pre-training on internet-scale text corpora via the next-token prediction objective.

2.2.2 Vision Encoders: SigLIP and DINOv2

Integrating visual inputs into language models requires a vision encoder that maps raw image pixels to a sequence of embedding vectors compatible with the language backbone’s representation space. Two families of vision encoders are particularly relevant to the present work.

SigLIP [23] is a contrastive vision-language encoder trained with a sigmoid loss function over image-text pairs. Unlike the softmax-based contrastive objective of CLIP [19], which requires computing pairwise similarities across all examples in a batch, the sigmoid formulation treats each image-text pair independently as a binary classification problem. This confers both computational advantages at large batch sizes and improved performance on tasks requiring fine-grained semantic alignment between visual and linguistic representations. SigLIP is the vision encoder employed in the VLM backbone of the proposed architecture.

DINOv2 [13] is a self-supervised vision encoder trained by distilling features from a teacher network using a combination of image-level and patch-level objectives. Because DINOv2 is trained without image-text supervision, its feature representations are less constrained by semantic label boundaries and exhibit notably strong spatial structure: patch-level features encode precise geometric and texture information at the scale of individual image regions. This property makes DINOv2 well-suited as a dedicated end-effector vision encoder, where fine-grained spatial features of the immediate manipulation workspace are more informative than high-level semantic

labels. The proposed architecture employs a successor model, DINOv3, in this role.

2.2.3 Multimodal Language Models and Instruction Following

Modern VLMs such as LLaVA [9] and GPT-4V [12] extend the language Transformer backbone to accept interleaved sequences of image tokens and text tokens. Visual inputs are first encoded by a vision encoder into a sequence of patch embeddings, which are then projected into the language model’s token embedding space via a learned linear or MLP adapter. The resulting multimodal sequence is processed by the language backbone in the same manner as a purely textual sequence, enabling the model to reason jointly about visual and linguistic content. Pre-training on large-scale image-text corpora, followed by instruction-tuning on curated demonstrations of instruction-following behaviour, yields models that can respond to complex compositional queries about image content with remarkable accuracy and generalisation.

The instruction-following capability of these models is directly relevant to robot manipulation: if a VLM can correctly parse an instruction such as “*put the banana in the green box*” and localise the referenced objects within a scene image, it provides a strong semantic grounding prior upon which an action generation module can be conditioned.

2.2.4 Object Localisation as Spatial Grounding

A dedicated line of research has investigated the extent to which VLMs can be trained to produce spatially grounded outputs in the form of explicit bounding box coordinates [2]. In this paradigm, the model is trained to generate token sequences such as **banana** `<loc12><loc25><loc67><loc89>`, where the `<loc>` tokens index a discretised coordinate grid. Training on this localisation objective has been demonstrated to substantially improve the spatial reasoning capacity of the underlying language backbone, with downstream benefits for tasks that require precise object-relative positioning—a property of direct relevance to manipulation planning. Accordingly, object localisation is incorporated as an auxiliary pre-training task in the proposed architecture.

2.3 DIFFUSION MODELS AND FLOW MATCHING

2.3.1 Denoising Diffusion Probabilistic Models

Denoising Diffusion Probabilistic Models (DDPMs) [4] define a generative model through a forward process that progressively corrupts a data sample $\mathbf{x}_0$ with Gaussian noise over T timesteps:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t I) \quad (2.2)$$

and a learned reverse process that denoises the corrupted sample step by step:

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)) \quad (2.3)$$

The reverse process is parameterised by a neural network trained to predict the noise ϵ added at each step, minimising the objective:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{\mathbf{x}_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2] \quad (2.4)$$

DDPMs have achieved state-of-the-art results in image generation and have been subsequently applied to robot action generation, where the data distribution to be modelled is the distribution of expert trajectory chunks rather than natural images [3].

2.3.2 Flow Matching

Flow Matching [8] is a more recent and computationally efficient alternative to DDPM-based generation. Rather than defining a multi-step noising process and learning to reverse it, flow matching directly learns a vector field $\mathbf{v}_\theta(\mathbf{x}, t)$ that transports samples from a simple source distribution (typically $\mathcal{N}(\mathbf{0}, I)$) to the target data distribution along straight-line paths in probability space:

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}_\theta(\mathbf{x}, t), \quad \mathbf{x}(0) \sim \mathcal{N}(\mathbf{0}, I), \quad \mathbf{x}(1) \sim \mathbf{p}_{\text{data}} \quad (2.5)$$

The vector field is trained by regressing against the conditional flow between pairs of noise and data samples. Compared to DDPM, flow matching admits fewer function evaluations at inference time, leading to substantially lower action generation latency—a critical property for real-time robot control. The proposed architecture employs a flow-matching objective within its Diffusion Transformer action expert.

2.3.3 Diffusion Transformer (DiT)

The Diffusion Transformer (DiT) [14] replaces the U-Net backbone traditionally employed in diffusion models with a Transformer architecture, enabling the model to condition on arbitrary context vectors via cross-attention or adaptive layer normalisation (adaLN). This architectural choice is particularly advantageous in the VLA setting, where the action expert must be conditioned on heterogeneous context signals of variable length—including VLM hidden state sequences and dense visual feature maps from the end-effector encoder. The DiT’s attention mechanism provides a principled and flexible mechanism for attending to these conditioning signals during the denoising process.

2.4 ACTION TOKENISATION

2.4.1 Discrete Action Representations

A central challenge in adapting language model backbones to robot control is the mismatch between the discrete token vocabulary of language models and the continuous, high-dimensional nature of robot joint trajectories. One family of approaches resolves this mismatch by discretising continuous trajectories into a finite vocabulary of action tokens, enabling the language backbone to predict actions autoregressively using the same next-token prediction objective employed during language pre-training. Early work in this direction employed naïve uniform binning of individual joint angles [1], but this approach incurs high quantisation error and an exponentially large vocabulary for multi-joint trajectories.

2.4.2 FAST+ Tokeniser

The Frequency-space Action Sequence Tokeniser (FAST) [17] addresses these limitations by representing trajectory chunks in the frequency domain via a Discrete Cosine Transform (DCT) prior to quantisation. The key insight is that robot trajectories are temporally smooth signals whose energy is concentrated in low-frequency DCT coefficients; retaining only the dominant coefficients before quantisation therefore preserves the essential

kinematic content of the trajectory with substantially fewer tokens and less quantisation error than spatial-domain methods. The FAST+ variant further improves upon the original FAST tokeniser through refinements to the codebook construction and decoding procedures, and has been empirically validated as the state-of-the-art discrete action tokeniser across a diverse set of manipulation benchmarks. Accordingly, FAST+ is adopted as the primary action tokenisation strategy in the proposed architecture, in accordance with current best practice.

2.4.3 Continuous State Representations

An alternative to discrete tokenisation is to represent robot state and action information as continuous vectors and inject them into the action model through learned conditioning mechanisms. Feature-wise Linear Modulation (FiLM) [15] is one such mechanism: it conditions a neural network layer on an external vector by learning affine transformation parameters (γ, β) that modulate the layer’s feature activations:

$$\text{FiLM}(\mathbf{h} \mid \mathbf{z}) = \gamma(\mathbf{z}) \odot \mathbf{h} + \beta(\mathbf{z}) \quad (2.6)$$

where $\mathbf{h}$ is the layer’s activation and $\mathbf{z}$ is the conditioning vector. FiLM injection is employed by the GR00T N1 architecture [10] to condition the action expert on proprioceptive state observations, and is noted in the proposed architecture as an alternative conditioning pathway to hidden state injection. The relative merits of these two conditioning strategies are empirically evaluated in Chapter 4.

2.5 RELATED VISION-LANGUAGE-ACTION ARCHITECTURES

2.5.1 OpenVLA

OpenVLA [5] is an open-source VLA model that established several widely adopted architectural conventions. It employs a dual-encoder vision backbone combining SigLIP for semantic visual features and DINOv2 for spatially structured patch features, whose outputs are concatenated and projected into the token embedding space of a pre-trained language model. Action outputs are produced by appending action token predictions to the instruction-grounded visual context, trained end-to-end via next-token prediction. OpenVLA demonstrated that fine-tuning a general-purpose VLM backbone on robot demonstration data yields a manipulation policy with substantially greater generalisation to novel objects and instructions than task-specific baselines. However, its reliance on a purely discrete autoregressive action output without a dedicated continuous action refinement module limits its trajectory smoothness and precision on tasks requiring fine-grained motor control.

2.5.2 GR00T N1

GR00T N1 [10] is NVIDIA’s open foundation model for humanoid robot control. It adopts a heterogeneous architecture in which a VLM backbone processes visual and linguistic inputs, and a separate action expert is conditioned on continuous state representations injected via FiLM. In contrast to the best practice of employing FAST+ for action tokenisation, GR00T N1 employs a customised tokenisation scheme optimised for the high-dimensional action spaces of full-body humanoid robots. The architecture targets large-scale, whole-body manipulation and locomotion tasks rather than the tabletop manipulation setting addressed in this thesis. GR00T N1 is nonetheless a relevant baseline and point of comparison, as it represents the most capable open VLA system currently available and embodies a comprehensive set of architectural design choices—including the frozen

backbone, FiLM-based conditioning, and multi-camera visual input—that directly intersect with the design space explored in the present work.

2.5.3 Diffusion-VLA

Diffusion-VLA [22] is a VLA architecture that incorporates a self-generated chain-of-thought reasoning module as an interpretable intermediary between language understanding and action generation. Before producing an action, the model generates a natural language rationale describing its interpretation of the scene and the sub-goals required to accomplish the instruction. This rationale is then provided as additional context to the diffusion-based action expert, enabling more interpretable and compositionally generalisable policies. Diffusion-VLA demonstrates that the language backbone of a VLA system can be leveraged not only as a feature extractor but as an active reasoning module, though the additional inference latency incurred by rationale generation is a practical limitation for time-sensitive control loops.

2.5.4 Knowledge Insulating VLA

The Knowledge Insulating VLA framework [16] proposes a pre-training strategy specifically designed to prevent the catastrophic forgetting of general visual and linguistic knowledge during robot-specific fine-tuning. The central observation is that naïve fine-tuning of a VLM backbone on robot demonstration data degrades the backbone’s generalisation capacity, because gradient updates from the manipulation task overwrite the broadly useful representations acquired during foundation model pre-training. The proposed remedy—selectively freezing the backbone and training only lightweight adapter modules alongside the action expert—is demonstrated to accelerate convergence, reduce data requirements, and improve generalisation to novel objects and instructions relative to full fine-tuning baselines. This finding motivates the frozen-backbone design adopted in the present architecture.

2.5.5 Summary and Research Gap

Table 2.1 summarises the key architectural and training characteristics of the surveyed VLA models in relation to the proposed architecture.

Table 2.1 Comparison of related VLA architectures. FM = Flow Matching, FS = Frozen Backbone, CT = Continuous Tokenisation, DT = Discrete Tokenisation.

Model	Vision Enc.	Action Gen.	Tokeniser	FS	End-Eff. Cam.	Localisation
OpenVLA	SigLIP + DINOv2	Autoregressive	Uniform bin	✗	✗	✗
GR00T N1	SigLIP	Diffusion (FM)	Custom (CT)	✓	✓	✗
Diffusion-VLA	SigLIP	Diffusion (FM)	FAST+ (DT)	✓	✗	✗
KILO-VLA	SigLIP + DINOv2	Autoregressive	FAST+ (DT)	✓	✗	✗
Ours	SigLIP + DINOv3	Diffusion (FM)	FAST+ (DT)	✓	✓	✓

The comparison reveals a consistent gap across all surveyed architectures: no existing model simultaneously employs a dedicated end-effector vision expert, FAST+ discrete action tokenisation, a flow-matching diffusion action expert, and object localisation as an auxiliary pre-training task within a frozen-backbone framework. The proposed architecture is designed to occupy this gap, synthesising the respective strengths of the surveyed systems into a unified design.

CHAPTER 3

RESEARCH METHODOLOGY AND SYSTEM DESIGN

This chapter presents the complete design and construction of the proposed Vision-Language-Action architecture. It details the rationale for each architectural component, the mathematical formulation of the training objectives, the pre-training curriculum, and the implementation specifications. The presentation follows the sequence of the system’s information flow: from raw sensor inputs through the foundation model and vision expert, to the action expert and the final continuous trajectory output.

3.1 RESEARCH METHODOLOGY OVERVIEW

The research methodology adopted in this thesis is structured around three sequential phases: architecture design, pre-training, and empirical evaluation. In the design phase, the system components are specified and their interactions formalised. In the pre-training phase, the foundation model and action expert are trained on large-scale robot demonstration datasets using the multi-task objectives described in Section 3.4. In the evaluation phase, the trained system is assessed on a suite of manipulation tasks under controlled conditions, with systematic ablation experiments isolating the contribution of each architectural and training choice. This chapter covers the first two phases; the evaluation phase is detailed in Chapter 4.

The methodology is grounded in the following core design principles:

- 1) **Frozen Backbone.** The VLM backbone is kept frozen throughout training to preserve the broadly generalisable visual and linguistic representations acquired during foundation model pre-training, in accordance with the findings of [16].
- 2) **Separation of Scales.** Overview-level semantic understanding and end-effector-level spatial precision are processed through separate, specialised encoders rather than a single shared encoder, enabling each module to be optimised for its respective representational requirement.
- 3) **Discrete-Continuous Hybrid Conditioning.** The action expert receives conditioning signals from both the discrete FAST+ action token pathway and the continuous hidden state pathway of the VLM backbone, combining the complementary strengths of both representations.
- 4) **Multi-Task Pre-Training.** The foundation model is trained on a curriculum that jointly optimises for object localisation, state description generation, and action token prediction, promoting spatial reasoning alongside action grounding.

3.2 SYSTEM ARCHITECTURE

3.2.1 Overall Architecture

The proposed system comprises three principal components: (1) a frozen VLM backbone responsible for high-level scene understanding and discrete action token prediction, (2) a trainable DINOv3-based Vision Expert that extracts fine-grained spatial features from the end-effector camera stream, and (3) a trainable Diffusion Transformer (DiT) Action Expert that generates continuous kinematic trajectories conditioned on the outputs

of the two preceding modules. Figure 3.1 illustrates the overall architecture and the information flow between components at inference timestep t .

Figure 3.1 Overall architecture of the proposed VLA system at a single inference timestep t . The frozen VLM backbone (cyan) processes the instruction and overview image; the Vision Expert (blue) processes the end-effector image; the Action Expert (purple) generates the continuous trajectory conditioned on both upstream modules.

Formally, at each timestep t , the system receives the following inputs:

- A natural language instruction $\mathcal{I}$ (e.g., “put the banana in the green box”)
- An overview RGB image $I_t^{ov} \in \mathbb{R}^{H \times W \times 3}$ of the workspace
- An end-effector RGB image $I_t^{ef} \in \mathbb{R}^{H' \times W' \times 3}$
- A textual state description s_t (e.g., “move to the banana on the bottom right of the table”)

and produces as output a continuous trajectory chunk $\hat{\mathbf{t}}_t \in \mathbb{R}^{N \times A}$, where N is the number of prediction timesteps in the chunk and A is the dimensionality of the robot’s action space.

3.2.2 VLM Backbone

The VLM backbone is implemented as a Qwen3 autoregressive language model augmented with a SigLIP vision encoder, with a combined parameter count of approximately 2 billion. The SigLIP encoder maps the overview image I_t^{ov} into a sequence of patch embeddings $\mathbf{V}^{ov} \in \mathbb{R}^{L_v \times d_v}$, where L_v is the number of patches and d_v is the visual embedding dimension. A learned linear projector $\mathbf{W}_{proj} \in \mathbb{R}^{d_v \times d_m}$ maps these embeddings into the language model’s token embedding space of dimension d_m .

The language model then processes the concatenated token sequence:

$$\mathbf{X}_t = [\text{tok}(\mathcal{I}), \mathbf{W}_{proj} \mathbf{V}^{ov}, \text{tok}(s_t), \mathbf{a}_t^{1:k-1}] \quad (3.1)$$

where $\text{tok}(\cdot)$ denotes the tokenised embedding of a text string and $\mathbf{a}_t^{1:k-1}$ are the previously predicted FAST+ action tokens at the current timestep. The backbone produces two outputs that are consumed by downstream modules:

- Discrete Action Tokens $\hat{\mathbf{a}}_t \in \mathcal{V}^K$:** the next FAST+ token is predicted autoregressively at each decoding step via a softmax output head over the action vocabulary $\mathcal{V}$.
- Hidden States $\mathbf{H}_t \in \mathbb{R}^{L_x \times d_m}$:** the sequence of hidden state vectors from the final Transformer layer, encoding the model’s joint contextual representation of the instruction, visual scene, and current state.

The backbone weights are frozen throughout all training stages. A lightweight LoRA adapter [?] may optionally be applied to the final self-attention layers corresponding to the **<start-of-action>** and **<end-of-action>** special tokens, enabling the backbone to learn task-specific action boundary representations without perturbing its general representations.

3.2.3 Vision Expert

The Vision Expert is implemented as a DINOv3 vision Transformer of approximately 86 million parameters. It receives the end-effector image I_t^{ef} and produces a sequence of patch-level feature embeddings:

$$\mathbf{F}_t^{\text{ef}} = \text{DINOv3}(I_t^{\text{ef}}) \in \mathbb{R}^{L_p \times d_p} \quad (3.2)$$

where L_p is the number of patches and d_p is the DINOv3 feature dimension. Because DINOv3 is pre-trained via self-supervised patch-level distillation without semantic label supervision, its patch features encode dense spatial geometry and texture information at fine resolution, providing the action expert with precise positional cues about the immediate manipulation environment that are absent from the VLM backbone’s higher-level semantic representations. The Vision Expert is trainable and is optimised jointly with the Action Expert during pre-training.

3.2.4 Action Expert

The Action Expert is a Diffusion Transformer (DiT) of approximately 300 million parameters that generates continuous trajectory chunks via a flow-matching objective. At each denoising step i of the generation process, the DiT receives:

- Noisy trajectory $\hat{\tau}_t^{(i)} \in \mathbb{R}^{N \times A}$ sampled from the current noise schedule
- Hidden states $\mathbf{H}_t$ from the VLM backbone, incorporated via cross-attention layers
- End-effector features $\mathbf{F}_t^{\text{ef}}$ from the Vision Expert, also incorporated via cross-attention layers
- Denoising timestep embedding $\phi(i)$

The DiT predicts the vector field $\mathbf{v}_\theta(\hat{\tau}_t^{(i)}, i)$ that transports the noisy sample toward the clean trajectory distribution:

$$\hat{\tau}_t = \hat{\tau}_t^{(0)} + \int_0^1 \mathbf{v}_\theta(\hat{\tau}_t^{(i)}, i) di \quad (3.3)$$

where the integral is approximated using an ODE solver with a fixed number of function evaluations. The cross-attention over $\mathbf{H}_t$ provides the primary semantic conditioning signal, while the cross-attention over $\mathbf{F}_t^{\text{ef}}$ provides the spatial precision signal. These two conditioning streams are processed through separate cross-attention sublayers within each DiT block, with their outputs summed before the subsequent feed-forward sublayer.

3.2.5 State Representation

Following the best practice identified in the literature review, the robot state $\mathbf{s}_t$ is represented as a **Text State**: a natural language description of the current manipulation sub-goal (e.g., “*move to a banana on the bottom right of the table*”). This representation is incorporated into the VLM input sequence (3.1) as a standard text token sequence, processed by the frozen backbone without any architectural modification. This contrasts with the Continuous State representation employed by GR00T N1 [10], which requires a dedicated FiLM injection pathway and additional trainable parameters at the backbone interface. The Text State is preferred here because it is natively

compatible with the backbone’s pre-trained token processing mechanism and imposes no structural modification on the frozen model.

3.3 ACTION TOKENISATION

3.3.1 FAST+ Discrete Tokenisation

Continuous trajectory chunks $\tau_t \in \mathbb{R}^{N \times A}$ are tokenised into a sequence of discrete tokens $\mathbf{a}_t \in \mathcal{V}^K$ using the FAST+ tokeniser [17]. The tokenisation proceeds in three stages:

1) **Frequency Transformation.** A Discrete Cosine Transform (DCT) is applied along the temporal dimension of the trajectory chunk, yielding a frequency-domain representation $\hat{\tau}_t = \text{DCT}(\tau_t)$. The energy of smooth robot trajectories is concentrated in the low-frequency DCT coefficients, enabling the trajectory to be compactly represented by retaining only the dominant K' coefficients.

2) **Scalar Quantisation.** Each retained DCT coefficient is independently quantised to the nearest centroid in a learned scalar codebook $\mathcal{C} \subset \mathbb{R}$, producing a sequence of $K' \times A$ discrete indices that are flattened and mapped to the action vocabulary $\mathcal{V}$ via a bijective mapping.

3) **Detokenisation.** At inference, the predicted discrete token sequence is decoded back to DCT coefficients, the discarded high-frequency components are set to zero, and an inverse DCT is applied to recover the continuous trajectory $\hat{\tau}_t$.

The FAST+ tokeniser is pre-fitted on the training dataset before any model training begins, and its parameters are held fixed throughout all subsequent training stages.

3.3.2 Special Action Boundary Tokens

Two special tokens, `<start-of-action>` and `<end-of-action>`, are added to the VLM vocabulary to demarcate the boundary of the action token sequence within the autoregressive generation process. The optional LoRA adapter is applied exclusively to the attention layers that process these boundary tokens, ensuring that the backbone learns appropriate action-transition representations while its general representations remain unperturbed.

3.4 PRE-TRAINING CURRICULUM

The pre-training stage trains the two trainable components—the Vision Expert and the Action Expert—using a two-track curriculum applied to the same demonstration dataset. The frozen VLM backbone participates in the forward pass but receives no gradient updates.

3.4.1 Foundation Model Track: Next-Token Prediction

The VLM backbone processes sequences of the following multi-task form:

$$\underbrace{I}_{\text{instruction}} \rightarrow \underbrace{\text{banana } \langle \text{loc12} \rangle \langle \text{loc25} \rangle \dots}_{\text{[1] object localisation}} \rightarrow \underbrace{s_t}_{\text{[2] state description}} \rightarrow \underbrace{\mathbf{a}_t \in \mathcal{V}^K}_{\text{[3] FAST+ action tokens}} \quad (3.4)$$

This multi-task sequence trains the backbone to jointly predict three categories of output tokens at each timestep:

[1] **Object Localisation.** The backbone is trained to predict the bounding box coordinates of task-relevant objects encoded as $\langle \text{loc} \rangle$ tokens indexing a discretised coordinate grid. This auxiliary task is motivated by the finding of [2] that spatial grounding training measurably improves downstream spatial reasoning capacity.

[2] **State Description.** The backbone is trained to generate the current natural language state s_t , grounding the language model in the temporal progression of the manipulation task across successive timesteps.

[3] **FAST+ Action Tokens.** The backbone is trained to predict the FAST+ discrete action token sequence $\mathbf{a}_t$ corresponding to the current trajectory chunk.

The training objective for this track is the standard cross-entropy loss:

$$\mathcal{L}_{\text{NTP}} = - \sum_j \log p_\theta(y_j | y_{<j}, \mathbf{x}_t) \quad (3.5)$$

where y_j denotes the j -th target token in sequence (3.4).

3.4.2 Action Expert Track: Reconstruction and Masked Autoencoding

In parallel, the Action Expert and Vision Expert are trained on a trajectory reconstruction objective. Given the ground-truth trajectory τ_t , a noisy version is constructed by interpolating linearly between the trajectory and a Gaussian noise sample at a random noise level $i \sim \mathcal{U}[0, 1]$:

$$\tilde{\tau}_t^{(i)} = (1 - i) \tau_t + i \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (3.6)$$

The Action Expert is trained to predict the conditional vector field that transports the noisy sample back toward the clean trajectory:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau_t, \varepsilon, i} \left[\left\| v_\theta(\tilde{\tau}_t^{(i)}, i \mid \mathbf{H}_t, \mathbf{F}_t^{\text{ef}}) - (\tau_t - \varepsilon) \right\|^2 \right] \quad (3.7)$$

An additional Masked Autoencoding (MAE) loss is applied to the FAST+ discrete action token sequence, wherein a random subset $\mathcal{M}$ of action tokens is masked and the Action Expert must reconstruct the masked tokens from the remaining context and the conditioning signals:

$$\mathcal{L}_{\text{MAE}} = - \sum_{k \in \mathcal{M}} \log p_\theta(a_k \mid \mathbf{a}_{\setminus \mathcal{M}}, \mathbf{H}_t, \mathbf{F}_t^{\text{ef}}) \quad (3.8)$$

The MAE objective encourages the Action Expert to develop a richer internal representation of action sequence structure, improving robustness to partial or noisy action token conditioning at inference time.

The total training objective is the weighted sum of the three losses:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{NTP}} + \lambda_{\text{FM}} \mathcal{L}_{\text{FM}} + \lambda_{\text{MAE}} \mathcal{L}_{\text{MAE}} \quad (3.9)$$

where λ_{FM} and λ_{MAE} are scalar weighting hyperparameters determined by grid search on the validation split.

3.5 DATASET COLLECTION PIPELINE

The training dataset used in this thesis is collected entirely in-house using a purpose-built data collection toolchain developed for the Dobot robotic arm [?]. This toolchain eliminates the dependency on third-party demonstration datasets and ensures that the collected data precisely matches the dual-camera input format and the 7-dimensional action space of the proposed architecture. The pipeline consists of four sequential stages: trajectory recording, trajectory post-processing, multimodal dataset creation, and data formatting.

3.5.1 Hardware Setup

The physical data collection platform comprises the following components:

- **Dobot robot arm** with a 6-DOF joint configuration and a tool digital output (DO) channel for gripper control, connected to the host workstation via TCP over a local Ethernet network. The dashboard command interface is accessed on port 29999; real-time joint state feedback is received on port 30004 at the native feedback rate.
- **Overview camera:** a USB RGB camera (OpenCV index 1) mounted above and behind the workspace to provide a global, scene-level view of the tabletop environment and the task objects.
- **End-effector camera:** an IP camera accessed via an HTTP MJPEG stream (<http://192.168.137.21:81/stream>), mounted at the wrist of the robot arm to provide close-range, proximity-level visual context of the gripper and the immediate manipulation target.

This dual-camera configuration corresponds directly to the two visual input streams consumed by the proposed VLA architecture: the overview image I_t^{ov} and the end-effector image I_t^{ef} , as defined in Section 3.2.2.

3.5.2 Stage 1: Trajectory Recording

Human demonstrations are recorded using the `record_trajectory()` method of the `DobotController` class. At the onset of recording, the robot is placed in drag (gravity-compensation) mode via the `StartDrag()` command, allowing the operator to guide the robot arm by hand through the desired manipulation motion. Joint states are sampled from the real-time feedback thread at a configurable rate (default: 50 Hz) for a configurable duration (default: 10 seconds), yielding a raw trajectory array of shape $(T, 7)$ where T is the total number of recorded frames.

Each row of the trajectory array encodes:

$$\mathbf{r}_t = [\Delta q_1, \Delta q_2, \Delta q_3, \Delta q_4, \Delta q_5, \Delta q_6, d_t] \quad (3.10)$$

where $\Delta q_j \in \mathbb{R}$ denotes the relative joint delta (in degrees) of the j -th joint with respect to the previous timestep, and $d_t \in \{0, 1\}$ is the binary state of the tool digital output channel, representing the open/closed state of the gripper. A deadband filter with threshold $\delta = 0.01$ degrees is applied during recording: if the maximum absolute delta across all joints falls below δ , the delta is set to zero, preventing sensor noise from being recorded as spurious motion.

3.5.3 Stage 2: Trajectory Post-Processing

The raw recorded trajectory is post-processed by the `post_process_trajectory()` method to remove redundant frames that would otherwise artificially inflate the dataset size and degrade training efficiency. Three criteria are jointly evaluated for each frame:

- 1) **Duplicate detection:** a frame is flagged as a duplicate if its maximum absolute difference from the preceding frame across all seven channels falls below 10^{-6} .
- 2) **Static movement detection:** a frame is flagged as static if the maximum absolute joint delta across the first six channels falls below 10^{-6} , indicating that no joint motion occurred.
- 3) **IO stability:** a frame is flagged as IO-stable if the tool digital output value is unchanged from the preceding frame.

A frame is removed if and only if all three conditions are simultaneously satisfied—a design choice that ensures frames in which the gripper state changes are always retained, even during otherwise static pauses in the demonstration. Additionally, all frames following the last retained movement frame are unconditionally preserved to maintain the natural termination behaviour of each demonstration. The post-processed trajectory is saved as a `.npz` file.

3.5.4 Stage 3: Multimodal Dataset Creation

The post-processed trajectory is replayed on the physical robot by the `dataset_creation.py` script to generate the multimodal dataset. The replay-and-capture procedure iterates through the trajectory in non-overlapping segments of fixed length $N_{\text{seg}} = 40$ timesteps. Before executing each segment, the following data are captured and saved:

- 1) **Robot state $s_t \in \mathbb{R}^7$:** the current absolute joint angles $[q_1, \dots, q_6]$ concatenated with the current digital output state, read directly from the feedback thread. Saved as a `.npz` file at `dataset/state/<task>/<index>.npz`.
- 2) **Overview image:** a single RGB frame captured from the overhead USB camera. Saved as a PNG file at `dataset/image/<task>/<index>.png`.
- 3) **End-effector image:** a single RGB frame captured from the wrist IP camera. The camera buffer is flushed by reading three consecutive frames before saving to avoid stale frames from internal buffering. Saved as a PNG file at `dataset/image/<task>/<index>_end.png`.

After saving the three data modalities, the corresponding trajectory segment—augmented with a look-ahead buffer of $N_{\text{buf}} = N_{\text{seg}}/2 = 20$ additional timesteps—is saved as a `.npz` file at `dataset/trajectory/<task>/<index>.npz`. The robot then executes the segment (without the buffer) using the `move_trajectory_smooth()` method at a control frequency of 80 Hz and a speed scale of 0.5, before advancing to the next segment.

3.5.5 Stage 4: Smooth Trajectory Execution

The `move_trajectory_smooth()` method implements a virtual governor controller for precise, low-latency trajectory replay. Rather than commanding the robot directly to each waypoint in the recorded trajectory, the method maintains a *virtual joint state* $\tilde{q}_t$ that moves from the robot's current position toward each successive waypoint at a controlled velocity, updated at the specified control frequency. The velocity profile follows a trapezoidal scheme:

$$\mathbf{v}_{t+1} = \text{clip}(\mathbf{v}_t \pm \alpha, \mathbf{v}_{\min}, \mathbf{v}_{\max}) \quad (3.11)$$

where $\alpha = 0.05$ deg/step is the acceleration increment, $\mathbf{v}_{\max} = 1.0 \times \eta$ deg/step is the speed-scaled maximum velocity (with $\eta \in [0, 1]$ the speed scale factor), and $\mathbf{v}_{\min} = 0.05$ deg/step is a minimum creep velocity enforced at the final waypoint to ensure arrival. The robot is commanded to the virtual joint state at each control step via `ServoJ()`, which bypasses trajectory planning and applies the command directly to the joint servos, enabling smooth motion at 80 Hz without introducing planning latency. For the final waypoint only, a braking profile is applied:

$$\mathbf{v}_{\text{brake}} = \sqrt{2\alpha \|\hat{\mathbf{q}}_t - \mathbf{q}_{\text{final}}\|_{\infty}} \quad (3.12)$$

ensuring that the robot decelerates smoothly to rest at the terminal position rather than abruptly stopping. A post-execution settling loop holds the final commanded position for up to 0.5 seconds until the physical joint error falls below a configurable threshold $\varepsilon = 0.5$ degrees.

The method additionally supports preemptive interruption via a `Queue` object: if a new trajectory signal arrives in the queue during execution, the current trajectory is immediately interrupted and the starting index of the interrupted segment is returned, enabling trajectory blending in the `inference()` method for real-time VLA deployment.

3.5.6 Dataset Summary and Formatting

The resulting dataset for each task episode consists of temporally aligned triplets of (overview image, end-effector image, robot state) paired with a corresponding trajectory fragment. Table 3.1 summarises the data format for a single recorded episode.

Table 3.1 Data format of a single episode in the collected dataset.

$N_{\text{seg}} = 40, N_{\text{buf}} = 20$.

Modality	Format	Shape / Size	Description
Overview image	PNG	$H \times W \times 3$	Workspace scene view
End-effector image	PNG	$H' \times W' \times 3$	Wrist-camera view
Robot state	<code>.npy</code>	$(7,)$	Joint angles + DO
Trajectory fragment	<code>.npy</code>	$(N_{\text{seg}} + N_{\text{buf}}, 7)$	Relative deltas + DO

The trajectory fragments are stored in relative delta format as per equation (3.10). Before being passed to the FAST+ tokeniser and the training pipeline, they are converted to absolute joint coordinates by cumulatively summing the deltas from the initial robot state, yielding absolute trajectory arrays of shape $(N_{\text{seg}} + N_{\text{buf}}, 7)$ that are subsequently chunked into overlapping windows of length $N = 16$ timesteps with stride 8 for training.

3.6 IMPLEMENTATION DETAILS

3.6.1 Hardware and Software

All training experiments are conducted on NVIDIA A100 80 GB GPUs. The VLM backbone is loaded in `bf16` precision with gradient checkpointing enabled to reduce memory overhead. The Vision Expert and

Action Expert are trained in `float32` precision. The codebase is implemented in Python using the PyTorch [?] deep learning framework, with the Hugging Face Transformers library [?] used for backbone weight management.

3.6.2 Optimiser and Learning Rate Schedule

Trainable parameters are optimised using AdamW [?] with a peak learning rate of 3×10^{-4} , weight decay of 10^{-2} , and gradient clipping at norm 1.0. A cosine annealing schedule with a linear warmup over the first 2% of training steps is applied. The optional LoRA adapter uses a separate learning rate of 10^{-4} and rank $r = 16$.

3.6.3 Hyperparameter Summary

The key hyperparameters of the proposed system are summarised in Table 3.2.

Table 3.2 Key hyperparameters of the proposed VLA architecture.

Hyperparameter	Value
VLM backbone	Qwen3 + SigLIP (≈ 2 B params)
Vision Expert	DINOv3 (≈ 86 M params)
Action Expert	DiT (≈ 300 M params)
Trajectory chunk length N	16 timesteps
Action space dimension A	7 (6-DOF + gripper)
FAST+ vocabulary size $ \mathcal{V} $	1,024
DCT frequency truncation K'	10 per action dimension
Flow matching steps (inference)	10
MAE masking ratio	0.4
Loss weight λ_{FM}	1.0
Loss weight λ_{MAE}	0.1
Batch size	256
Training steps	200,000
Peak learning rate	3×10^{-4}
LoRA rank r (optional)	16

3.7 INFERENCE PROCEDURE

At inference time, the system operates in a receding-horizon fashion. At each control timestep t , the VLM backbone performs a single autoregressive forward pass over the input context (3.1), producing hidden states $\mathbf{H}_t$ and the discrete action token sequence $\hat{\mathbf{a}}_t$. Simultaneously, the Vision Expert processes the current end-effector image to produce $\mathbf{F}_t^{\text{ef}}$. The Action Expert then denoises a Gaussian noise sample $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}^{N \times A})$ conditioned on $\mathbf{H}_t$ and $\mathbf{F}_t^{\text{ef}}$ over 10 ODE integration steps, yielding the predicted trajectory chunk $\hat{\mathbf{t}}_t$. The first $N/2$ steps of $\hat{\mathbf{t}}_t$ are executed on the robot; the remaining steps are discarded, and the process repeats at the next timestep with updated observations. This receding-horizon execution strategy balances trajectory smoothness with reactivity to environmental changes.

CHAPTER 4

REFERENCE AND CITATION

Citation refers to the reference documents used in thesis writing. It is a way to inform the sources of the text, ideas, or any content that is not originally created by the author. This is done as a tribute to individuals or organizations who own the intellectual property of the ideas or the data. It also serves the convenience of readers who wish to know more details from the original source to be able to trace accurately. Citations can either summarize the original content or can be a direct quotation, maintaining the original writing format. To ensure accuracy, all information in the reference list must be precise and thoroughly checked for their sources.

Citations come in two main formats:

In-text citation using the name-year system.

In-text citation using the numerical sequence system.

At the end of the thesis, all the documents and references cited can be organized alphabetically according to the authors' names in a **bibliography** or in numerical order according to the reference numbers in a **reference** list.

4.1 IN-TEXT CITATION IN THE NAME-YEAR SYSTEM

Citations should include the author's name and the publication year, with the option to include page numbers separated by a colon (:).

Author names in Thai documents should include both first and last names, while in foreign language documents, only the last name should be used.

The citation format may vary depending on the sentence structure, either placed at the beginning or end of a sentence.

Example (Author mentioned at the beginning of the sentence):

Suttalak Ampanswong (2521: 25) explains that information refers to various knowledge, news, and data.

Good (1973: 112) defines commitment as an individual's expression of love and concern.

Example (Author mentioned at the end of the sentence):

...managers must have the ability to manage people to foster cooperation and collaboration in responsive and well-coordinated work, leading to the efficient achievement of organizational objectives (Uthai Boonprasert, 2531: 23).

In citations, there may be different cases as follows:

1. Single Author

Mayuree Chaisawat (2538: 86)...

Heyes (1964)...

2. Two Authors

Prayad Jantachomphu and Prasopas Aksonmat (2518: 24)...

Macaulay and Berkowitz (1978: 4)...

3. More than Two Authors

Sanan Jittrasuk et al. (2532: 21-25)...

Bradley, S. et al. (1983: 23-25)...

4. Authors from an Institution, Organization, Corporation, or Agency

From the meeting of education institution managers under the Department of Vocational Education (Department of Vocational Education, 2531) on the topic "Human Resource Needs."

...and environmental pollution, termed as 'air pollution' or the occurrence of air pollution (Department of Environmental Quality Promotion, 2539: 118).

5. Multiple Titles by the Same Author in the Same Year

Sutthas Yaksan (2529a)...

Heyes (1964c)...

6. Multiple Citations within the Same Topic from Multiple Sources

...(Yawonuch Sengyont, 2525a; Supada Intranukul, 2525)...

...(Kartner, 1973; Kartner and Russel, 1975)...

7. No Author Identified

Use 'Anonymous'

Anonymous (1973)...

8. Quotations Longer Than 3 Lines

Should be separated from the main text with a one-line space above and below. The left and right margins of both sides should be indented 8 character spaces, followed by the name-year citation in parentheses, without quotation marks ("").

Example:

...The personnel in the organization are the heart of the organization, a vital force that propels the organization forward. As Somyasit Naweeakarn stated,

(Leave 1 line blank)

'Internal conflicts within an organization can be designed or managed in various ways. Conflicts can yield positive and negative results. The positive aspect is that conflicts lead to the exploration of better outcomes, making the organization operate more efficiently. However, if conflicts are excessive, they can negatively impact the personnel in the organization and impede the achievement of the organization's goals' (Somyasit Naweeakarn, 2530: 15).

9. Electronic Information and Communication

For electronic sources such as CD-ROM, Internet, use the previously mentioned citation methods.

4.2 IN-TEXT CITATION IN THE NUMERICAL SEQUENCE SYSTEM

In this referencing system, citations are numbered sequentially in the order they appear in the text, starting from the first chapter to the last. The reference numbers are enclosed in square brackets at the end of the relevant text. If a reference is repeated, the same number is used.

Example:

Due to the necessity of frequency-based decoding, various transformations must be applied, including essential conversions [1]. Apart from other computations, the prime factor algorithm is employed for

decoding, reducing the computational load when combined with the Short Length Algorithm [2]. In practical implementation, the application of these methods requires careful consideration...

Alternatively, Berlekamp's technique has the capability to generate both polynomials [2] . However, the process becomes more intricate in the presence of erasures. Blahut [3] has detailed the procedure for obtaining the error-erasure locator polynomial. This involves initializing the error locator polynomial with the erasure locator polynomial. He has also demonstrated that the error evaluator polynomial and the derivative of the error locator polynomial can be iterated within those $2t$ iterations [4].

CHAPTER 5

BIBLIOGRAPHY AND REFERENCES

The writing of a bibliography or references should adhere to one system that aligns with the chosen citation method. Use 'Bibliography' when citing in-text with the name-year system and use 'References' when citing in-text with the numerical sequence system. The use of bibliography management software, such as Endnote, Mendeley, Zotero, etc., is permitted and encouraged.

5.1 GUIDELINES FOR WRITING BIBLIOGRAPHY

1. Print the word Bibliography in the center of the page in bold, size 24 points.
2. Arrange the entire reference list together, alphabetically based on the first letter of each reference, following the alphabetical order as per the dictionary.
3. For theses written in Thai, list the bibliography in the Thai language first, followed by the English language bibliography. For theses written in English, list the English language bibliography first, followed by the Thai language bibliography.
4. Begin printing the bibliography at the left edge of the paper. If the entry does not fit on one line, start a new line indented by 7-character spaces. Begin printing the 8th character if the entry extends to the second line. If not completed within two lines, continue onto lines 3-4 starting from the same position as line 2 until the end of the entry. When starting a new entry, align it with the left edge of the paper without skipping a line.

5.2 GUIDELINES FOR WRITING REFERENCES

1. Print the word References in the center of the page in bold, size 24 points.
2. Arrange the reference documents in numerical order, according to the numbers assigned within parentheses referring to the content of the thesis.
3. There is no need to separate by language or document type.
4. Print the number of each entry at the left edge of the paper. If the entry does not fit on one line, start a new line indented by 7-character spaces. Begin printing the 8th character if the entry extends to the second line. When starting a new entry, align it with the left edge of the paper without skipping a line.

5.3 METHOD OF LISTING BIBLIOGRAPHY

5.3.1 Authors

1. The author's name should not include any titles such as Mr., Mrs., Miss, Dr., etc.
2. For entries in the Thai language, use the author's first name followed by the last name.
3. For entries in English, use the last name, followed by a comma and the abbreviated first and middle names in sequence.
4. If the author holds a doctorate (Ph.D.), professorship, or other title, include it after the name, separated by a comma.

5. Use the name as it appears in the document for authors with honorific titles.
6. If there are two authors, use the name of the first author followed by and between the names of the two authors.
7. If there are more than two authors, list all authors.
8. For authors using pseudonyms, use the pseudonym as it appears in the document.
9. For translated documents, list the name of the translator first, followed by the phrase "translated from" and the original author's name. If the original title is unknown, use the phrase "translated by" or "translated and compiled by."
10. For documents attributed to an organization, ministry, or department, use the name of the organization as the author. In the case of documents issued under a specific department within a ministry or subordinate to a department, even if the ministry's name is mentioned, use the department's name as the author. Include the name of the subordinate unit in the publisher section.
11. For documents with only an editor or compiler, use the editor's name followed by a comma and the word editor or compiler after the name of the author.
12. For documents with no stated author or conference proceedings, use the title of the document as the first entry.

5.3.2 Article Titles

1. Use the title as it appears in the document, enclosed in quotation marks.
2. For English titles, capitalize the initial letter of each word, except for conjunctions, articles, and prepositions, unless they are the first word.
3. If there is a scientific name in Latin for plants, animals, etc., insert it in *italics* or underline it.

5.3.3 Book or Journal Titles

1. Print book or journal titles in bold.
2. For international journals, you may use the correct abbreviation of the journal name.
3. For book series, if referencing a specific volume, include only that volume. For example, if referring to volume 3, and if citing multiple non-consecutive volumes, indicate each volume number with commas in between, such as Volume 1, 3, 5. If citing all volumes in the series, state the total number of volumes, such as 5 volumes or 5 vol. Underline or print in bold (either one) continuously from the title.

5.3.4 Editions

1. For the first edition or first printing, it is not necessary to specify in the bibliography.
2. Specify the edition number for subsequent editions, as indicated in the printed material. For example, 2nd Ed., 2nd printing, 2nd, rev. ed., or 2nd, expanded ed.

5.3.5 Place of Publication and Publisher

1. For Thai-language references, include the name of the province as the place of publication. For Bangkok, use the term Bangkok.
2. If the publisher has offices in multiple cities and the names of those cities appear in the document, use the first city mentioned as the place of publication.

3. List the publisher's name without including terms like Publisher, Company Limited, "Publisher," "Co—Inc.," or "Co..Ltd."
4. If the document does not mention the publisher's name, list the name of the printing house with the term Printing House.
5. For publishers associated with universities or organizations, provide the full name, such as Thammasat University Press.
6. If the printer is a government or private organization, use the organization's name as the publisher.
7. If the name of the person responsible for printing is not stated, use 'n.p.' (no place of publishing) in English.

5.3.6 Year of Publication

1. Include the year of publication in numeric format, as it appears in the document.
2. If the publication year is not mentioned, use 'n.d.' (no date) in English.

5.3.7 Example of Bibliography Format

The specifications for referencing in the bibliography format are as follows:

1. Variables in different listings, such as author, day, month, year, and publisher, should use the regular Courier PC format.
2. Variables that need to be printed in bold should use the Courier Upright Bold format.
3. Terms that need to be printed in italics, such as 'ผู้แปล' (translator) or 'โดย' (by), should use the regular Arial Unicode MS format.
4. Replace single-letter spacing with the '/' symbol.
5. In cases where a single line is not sufficient, start the second line with the eighth character, leaving a space of 7 characters.
6. Other symbols such as '.', ',', ' ', ':', '-', '[]', should be printed as specified.

5.3.7.1 Book

Format:

Author./Publication Year./Title./Volume (if applicable)./Edition (if applicable)./City of Publication./Publisher.

Examples:

Single Author:

Sumner, M. 1990. Computer: Concept and uses. 2nd ed. New York: McGraw-Hill.

Two Authors:

Gilbert, A. and Gnglir, J. 1982.**Cities Poverty and Development Urbanization in the Third World**. London: Oxford University Press.

Multiple Authors:

Jackson, M.H. Stewart, D. and Steven, G. 1991.**Environmental Health Reference Book**. Oxford: Butterworth Heinemann.

Authors with Pseudonyms:

น.ม.ส. (นามแฝง). 2496. นิทานของ น.ม.ส. พิมพ์ครั้งที่ 3. พระนคร: คลังวิทยา.

5.3.7.2 Translated Books

Format

Author./Year BE. Publication Year./Book Title./Volume (if any)./Translated by Translator./Edition (if any)./City of Publication:/Publisher.

Or

Author./Year BE. Publication Year./Book Title./Volume (if any)./Translated from **Original Title**./by Translator./Edition (if any)./City of Publication:/Publisher.

Example

Davenport, J.H. 1993.**Computer Algebra**. Translated by Davenport, A and Davenport, J.H. 2nd ed. Great Britain : Academic Press.

Shafarevich, I.R. 1994. Basic Algebraic Geometry 1. translated from **Osnovy Algebraicheskoy Geometrii, tom 1**. by Reid, L. 2nd ed. New York : Springer.

5.3.7.3 Translated Books with Unspecified Original Author in the Original Language

Format

Translator Name./Translator./Year BE. Publication Year./Book Title./Volume (if any)./Edition (if any)./City of Publication:/Publisher.

Example

Duang Bunnak, Translator. 2511. Economics. 2nd ed. Bangkok: Thammasat University.

Apichat Thaowto and Serm Sakkukul, Translators. 2526. Basic Knowledge of Rice Cultivation. Bangkok: Kasetsart University.

5.3.7.4 Books Authored by Organizations

Format

Organization Name./Year BE. Publication Year./Book Title./Volume (if any)./Edition (if any)./City of Publication:/Publisher.

Example

The Bank of Thailand. 1992. 50 Years of the Bank of Thailand 1942-1992. Bangkok : Amarin Printing Group.

5.3.7.5 Editors or Compilers

Format

Editor Name./Editor (Compiler)./Year BE. Publication Year./Book Title./Volume (if any)./Edition (if any)./City of Publication:/Publisher.

Example

Friedman, E.G., editor. 1995. Clock Distribution Networks in VLSI Circuits and Systems. New York : IEEE Press.

5.3.7.6 Conference Proceeding or Articles from Books

Format

Author of the Article./Year BE. Publication Year./ "Article Title (Conference Report)."/Page/Page Range./In/Editor (Compiler)./Book Title./Volume (if any)./Edition (if any)./City of Publication./Publisher.

Example

Niwa, E. 1992. "Chemistry of surimi gellation." 115-118. in Lainier, T.C. and Lee, C.M. Surimi Technology. New York : Marcel Dekker

Goemans, M.X. and Williamson, D.P. 1995. "The Primal-dual Method for Approximation Algorithms and its Application to Network design Problems." 69-87. in Hochbaum, D. Approximation Algorithms for NP-hard Problems. Boston : PWS.

5.3.7.7 Journal Articles

Format

Author./Publication Year./ "Article Title."/Journal Name./Volume(Issue)/:Page Range.

Bentley, M Lee. 1986. "The Role of Backcountry Experience in Middle School Environmental Education," Dissertation Abstracts Internation. 46(10) : 2900-A-2901-A.

Choomchuay, S. 1994. "On the Implementation of Finite Field Operation." Ladkrabang Engineering Journal. 11(1) : 7-16.

Hitchcock, R.B. et. al. 1982. "Timing Analysis of Computer Hardware." IBM J. Res. Develop. 26(1) : 100-105.

Klein, P. et. al. 1995. "An approximate max-flow min-cut relation for undirected multi-Commodity flow, with applications." Combinatorica. 15(2) : 187-202.

5.3.7.8 Thesis and Dissertation

Format

Author./Year BE. Publication Year./ "Thesis Title."/Degree Name (Full).

Example

Choomchuay, S. 1993. "Algorithm and Architecture for Reed-Solomon Decoding." Ph.D.Thesis of University of London and the Diploma of Membership of the Imperial College.

Burns, B. 1991. "Performance Analysis and Optimization of Asynchronous Circuits." Ph.D. Dissertation, California Institute of Technology.

5.3.7.9 Reviews

Format

Reviewer Name./Review of/Title of the Reviewed Work./by/Author Name./In/Book Title (where the review appears)./Edition (if any)./Page Range.

Example

Chamnan Nakprasom. Review of the book Lai Suea Siam by S. Siwarak. Social Perspectives Journal, Volume 5(1), Issue 1 (June-August 2510): 139-141.

5.3.7.10 Articles in Encyclopedias or Dictionaries

Format

Author.//”Article Title.”//Encyclopedia or Dictionary Name.//Volume (Publication Year).://Page

Range.

Example

Prapat Trinanong. 2513. ”Chor Fa.” Thai Encyclopedia by the Royal Institute, Volume 10: 6076-

6080.

5.3.7.11 Newspaper Articles

Format

Author.//Publication Year, Date, Month.//”Article Title.”//Author of the Article.//Newspaper Name.//Page/Page

Range.

Example

Goleman, D. 1985, 21 May. ”New Focus on Multiple Personality.” New York Times. Pages c1-c6.

5.3.7.12 News Monitoring and Facts Checking in Newspapers Over a Period

Format

Newspaper Name.//Date, Month//Year of Publication - Date, Month//Year of Publication.

Example

Thai Rath. January 4, 2538 - February 5, 2538.

5.3.7.13 Newsletters, Photocopies, and Unpublished Documents

Format

Author.//Year BE. Publication Year.//”Document Title.”//Edition (if any).//City of Publication://Publisher.//Document Copy.

Example

Sakallah, K.A. et. al. 1989. ”Analysis and Design of Latch-Controlled Synchronous Digital Circuits.” Ann Arbor, Mi, : Dept. of EECS University of Michigan Technical Report CSE-TR-31-89.

5.3.7.14 Government Publications and Other Official Documents

Format

Organization Name.//Publication Year.//Publication Title.//Edition (if any).//City of Publication://Publisher.

Example

Office of the Secretary-General of the House of Representatives. 2522. Reports of the 8th-9th Representative Assembly Regular Session Volume 3. Bangkok: Printing Office of the Cabinet.

5.3.7.15 Interviews

Format

Interviewee Name./Interviewer./Date, Month Year of the Interview./Interviewer Name./Interview Title./Interview Location.

Example

Manas Sangwarasilp Interviewed, April 18, 2541. Somsak Chumchuay, Interviewer. Interview on the Improvement Guidelines for Graduate Programs of the Institution. Graduate School, King Mongkut's Institute of Technology Ladkrabang.

5.3.7.16 Audiovisual Materials

1) Slides, Filmstrips, Videos, Tape Recordings, Compact Discs

Format

Creator./Year BE. Production Year./Title./[Type of Medium]./Place of Production:/Producer.

Example

Charoen Prokpun Ltd. 2525. Chicken Farming. [Video]. Bangkok: Charoen Prokpun Ltd.
 Alan W.B. Compiler. 1984. Using the Oscilloscope. [Slide]. London : The Slide Centre.
 RCA Records. 1984. Test Compact Disc ; Extensive Test for Audio Equipment. [Compact Disc].
 Germany : RCA Ltd.

2) Maps

Format

Name [Map Type]./Publication Year./Publication Title./Publisher.

Example

Scotland [Photographic map]. Washington (DC) : National Geographic Society (US.)

5.3.7.17 Patents

Format

Patentee./Invention Name./Country of Patent./Patent Number./Date of Patent Grant.

Example

Kamder, A.D. Method and Apparatus for Constraining the Compaction of Components of a Circuit Layout. U.S patent no. 5636132, June 1997.

5.3.7.18 Electronic Media

Format

Author./Publication Year./Title./[CD-ROM]./Place of Production:/Producer.

or

Author./Publication Year (or Retrieval Date)./Title./[Online]./Available:/Access Method and Data Location.

Examples

Daniel H., Anghileri. 1995. Secondary Mathematics and Special Education Needs. [CD-ROM].
 New York: Casell.

Noam E.M. 1994 Telecommunication Policy Issue for the Next Century. [Online]. Available: Gopher://198.80.36.../global/telecom.txt.

Texas Instruments. 1998. Speech synthesis Processors. [Online]. Available: <http://www.ti.com/sc/docs/msp/speech/dex.htm>.

5.3.8 Example Formats for References Citation

5.3.8.1 Book

Format

Author./Book Title./Edition (if any)/City of Publication./Publisher./Publication Year BE.

or

Editor Name./Editor (Compiler)/Book Title./Edition (if any)/City of Publication./Publisher./Publication Year BE.

Example

[1] Richard E. Blahut. Theory and Practice of Error Control Codes. Reading : Addison-Wesley publishing Company, Inc.1984.

[2] Judith S.Van ALSTYNE. Professional and Technical Writing Strategies. International Edition. New JERSEY : Prentice-Hall, Inc. 1986.

[3] Jose E. Franca, Yannis Tsividis, Editors. Design of Analog-Digital VLSI Circuits for Telecommunication and Signal Processing. 2nd ED. New Jersey : Prentice-Hall, Inc.1994

[4] Microsoft. Hardware Design Guide for Window 95 ; A Practical Guide for Developing Plug and Play PCs and Peripherals. Washington D.C. : Microsoft Press. 1994.

[5] Microelectronics Group, Lucent Technology. Field-Programmable Gate Arrays [data Book]. LUCENT Technology. October 1996. Pp.2-36-2-40.

5.3.8.2 Articles from Journals, Books, or Conference Proceedings

Format

Author./"Article Title."//Journal Name./Volume,/Issue Number./Month Year of Printing./Page/Page Range.

Example

[1] Choomchuay S. "On the Implementation of Finite Field Operations" Ladkrabang Engineering Journal, vol.11,no, 1, June 1984. pp.7-17.

[2] Shao M., Reed S. "On the VLSI Design of a Pipeline Reed-Solomon Decoder Using Systolic Arrays." IEEE Trans. On Comput., vol. C-37, no. 10, Oct. 1988. pp. 1273-1280

[3] Sukiyaama Y., Kasahara M., Hirasawa S. and Namekawa T. "A Method for Solving key Equation for Decoding Coppa Codes.", Inf. And Cont., vol.27, 1975. Pp. 87-99

[4] Choomchuay S., Arambepola B. "An Algorithm and a VLSI Architecture for Reed – Solomon Decoding" Proc. IEEE-ISCAS, San Diego, USA, May, 1992. Pp.2120-2123

5.3.8.3 Thesis and Dissertation

Format

Author.// "Thesis Title."// "Full Degree Name."//Year of Publication BE.

Example

[1] Choomchuay S. "Algorithm and Architecture for Reed-Solomon Decoding." Ph.D. Thesis of University of London and the Diploma of Membership of the Imperial College. 1993

5.3.8.4 Patents

Format

Patent Holder.//Invention Title.//Country of Patent.//Patent Number.//Date of Patent Registration (Day, Month, Year).

Example

[1] Kandar A.D. "Method and Apparatus for Constraining the Compaction of Components of a Circuit Layout." U.S patent no. 5636132, June 1997

5.3.8.5 Visual Materials and Other Media

Format

Producer.// "Title." // [Type of Media]//Place of Production.//Producer.//Year of Production BE.

Example

[1] RCA Records. "Test Compact Disc ; Extensive Test Audio Equipment." [Compact Disc]. Germany : RCA Ltd. 1984.

[2] Alan W.B. Compiler. "Using the Oscilloscope." [Slide]. London : The Slide Center. 1984.

5.3.8.6 Electronic Media

Format

Author.// "Title." // [CD-ROM]// Place of Production.//Producer.//Year of Release BE.

Or

Author.// "Title." // [Online]//Available at://Access Method and Source.//Year of Publication BE (or retrieval).

Example

[1] Daniel H., Anghileri. "Secondary Mathematics and Special Education Needs." [CD-ROM]. New York : Casell.1995

[2] Noam E.M. "Telecommunication Policy Issue for the Next Century." [Online]. Available : gopher://198.80.36.../global/telcom.txt. 1994.

[3] Texas Instruments. "Speech Synthesis Processors." [Online]. Available : <http://www.ti.com/sc/does/msp/speech/dex.htm>. 1998.

Bibliography

- [1] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-1: Robotics transformer for real-world control at scale. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [2] Jinkun Chen et al. Learning to localize objects improves spatial reasoning in visual-LLMs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. Supports object localisation as a VLM pre-training auxiliary task.
- [3] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2024. Establishes the dual-encoder SigLIP + DINOv2 backbone convention.
- [6] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [7] Sergey Levine, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016.
- [8] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] NVIDIA. GR00T N1: An open foundation model for generalist humanoid robots. Technical report, NVIDIA Corporation, 2024. Demonstrates FiLM-based continuous state conditioning and customised action tokenisation.
- [11] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [12] OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023.
- [13] Maxime Oquab, Timothée Darcet, Théo Moutakanni, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2023. Foundation for the DINOv3 vision expert used in the proposed architecture.
- [14] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

- [15] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [16] Karl Pertsch et al. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. *arXiv preprint*, 2025. Pre-training best practice reference from the architecture document.
- [17] Karl Pertsch, Robin Strudel, et al. FAST: Efficient action tokenization for vision-language-action models. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2025. State-of-the-art discrete action tokeniser; referred to as FAST+ in this thesis.
- [18] Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1989.
- [19] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2021.
- [20] Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [22] Wenxuan Wen et al. Diffusion-VLA: Generalizable and interpretable robot foundation model via self-generated reasoning. *arXiv preprint*, 2024. Introduces chain-of-thought reasoning as an interpretable action prior.
- [23] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

Appendix A
GUIDELINES AND PROCEDURES FOR THESIS WRITING

Thesis writing is considered a crucial part for graduate students after completing coursework at a certain level. Students must plan their thesis writing process until the comprehensive thesis examination is completed.

Procedures for Requesting Approval of Thesis Topics and Scheduling Thesis Examinations:

1. Graduate students have the right to propose thesis topics as soon as they become enrolled in the graduate program.

2. Students must submit a request for approval of the thesis topic to the graduate school of their respective faculty.

3. The approval of the thesis topic and the thesis outline is within the authority of the dean, endorsed by the academic affairs committee.

4. Students must receive approval of the thesis topic and the outline (if any) before the final thesis examination.

5. Any changes before the thesis examination must be handled as follows:

5.1 **Change of thesis topic or outline:** Students must submit a request to the dean, with approval from the thesis advisor and endorsement from the academic affairs committee.

5.2 **Change of thesis advisor:** Students must submit a request to the dean of the respective academic unit, and the dean has the authority to approve, with endorsement from the academic affairs committee.

5.3 In cases where the thesis advisor loses their position within the academic unit, the student must find a new advisor, and the work specified by the owner of the work, including the student's name, the name of the thesis advisor, the name of the academic unit, and the name of the institution, should be used to support the thesis examination application.

6. The thesis examination is the final stage, and students must comply with the conditions and requirements of each program in their respective academic units. Students can only take the thesis examination when:

6.1 The thesis advisor believes that the student is ready for the examination. The student must notify the examination committee and submit a draft of the thesis to the specified number of graduate students, faculty, and the examination committee no less than two weeks in advance.

6.2 For master's degree examinations, the branch chairman and the course coordinator must conduct the thesis examination committee, endorsed by the academic affairs committee. The committee should consist of five members, including the thesis advisor, one graduate faculty member proposed by the thesis advisor, and three members, including an outside faculty member who is an expert, proposed by the academic affairs committee. The committee must appoint the external committee member as the committee chairman.

6.3 For doctoral degree examinations, the dean should appoint the thesis examination committee, endorsed by the academic affairs committee. The committee should consist of five members, including the thesis advisor, one faculty member proposed by the thesis advisor, and three members, including an outside faculty member who is an expert, proposed by the academic affairs committee. The committee must appoint the external

committee member as the committee chairman.

AUTHOR BIOGRAPHY



Name Mr. First Name Last Name

Date of Birth January 1, 1990, in Bangkok

John Doe was born in Bangkok, Thailand, on January 1, 1995. He received his Bachelor of Engineering degree in Robotics and AI Engineering from KMITL in 2017. He is currently pursuing a Master of Engineering degree in Robotics and AI Engineering at KMITL. His research interests include machine learning, computer vision, and robotics.